

Содержание

1. Введение	2
2. Зависимости	4
3.1. Интеграция с CMake	5
3.2. Интеграция с QMake	8
3.3 Упаковка скомпилированной библиотеки в .deb пакет	9
4. Класс Config	10
4.1. Формат JSON	12
4.2. Формат INI	14
4.3. Формат XML (TODO)	16
4.4. Формат YAML (TODO)	17
5. Работа с комментариями	18
5. Сетевое взаимодействие	23
6. Дополнительные функции-утилиты	24

1. Введение

Библиотека SimpleAPI задумывалась как инструмент для удобной работы с файлами конфигураций с возможностью использования комментариев внутри файлов. А также передача конфигов по сети в указанном формате(**TODO**) (без комментариев) между программами.

То есть подразумевается несколько сценариев взаимодействия с конфигами:

- программа-человек: файлы с возможностью комментирования
- программа-программа: простая, с точки зрения программиста, упаковка значений в строку конфига, её передача до другого компьютера через сокет и простая распаковка в соответствующие значения на другом компьютере; можно отправлять как JSON-объекты, так и простые потоки байтов - SimpleAPI Network сделает за программиста всё остальное.

Реализовано:

- полная поддержка формата JSON
- полная поддержка формата INI - стандарта не существует, но общие концепции сторонних реализаций сохраняются (свободно конвертируется в JSON и обратно)
- возможность указать стиль комментариев (символы границы 1-2 знака для однострочных и 1-3 знака - для многострочных), их чтение из файла и запись в него
- сетевое взаимодействие сервер-клиент в формате UDP с автоматической фрагментацией и дефрагментацией пакетов, перепосылкой недоставленных фрагментов (гарантированная стабильность только в локальной сети)

В планах:

- поддержка XML (начат поиск решения)

- поддержка YAML
- шифрование при отправке по сети

2. Зависимости

Минимальные требования к системе

ОС: Linux

Стандарт языка: C++11

Библиотеки

Проект написан собственными средствами стандарта языка C++ и зависит только от библиотеки `pthread` для класса `SocketThread`.

3.1. Интеграция с CMake

Минимальная версия CMake для сборки библиотеки 3.5.

Библиотеку можно добавить несколькими способами:

Как самостоятельный CMake проект

```
cd <SimpleAPI_dir>
mkdir -p build
cd build
cmake [-DCMAKE_BUILD_TYPE=Debug] .. # флаг -DCMAKE_BUILD_TYPE указывается
опционально
make -j$(nproc)
```

В директории build появится директория lib с собранной библиотекой в формате .so и директорией include со всеми заголовочными файлами библиотеки.

Как часть другого CMake проекта (add_subdirectory)

В составе вызывающего CMakeLists.txt следует указать следующий код CMake:

```
set(SIMPLE_API_MAIN_DIR ${CMAKE_CURRENT_LIST_DIR}/../..) # указать путь до
базовой директории библиотеки
include(${SIMPLE_API_MAIN_DIR}/utils.cmake) # для доступа к макросам библиотеки

# код для определения типа сборки
if(CMAKE_BUILD_TYPE STREQUAL "Release")
    set(SimpleAPI_lib_name SimpleAPI)
else()
    set(SimpleAPI_lib_name SimpleAPId)
endif()

add_subdirectory(${SIMPLE_API_MAIN_DIR}/SimpleAPI
${CMAKE_CURRENT_BINARY_DIR}/${SimpleAPI_lib_name})
```

```
# код для создания таргета основного приложения
add_executable(...)

# предполагается, что имя таргета совпадает с именем проекта (PROJECT_NAME)
# добавление зависимости от библиотеки для созданного таргета
target_link_libraries(${PROJECT_NAME} PUBLIC ${SimpleAPI_lib_name})
target_include_directories(${PROJECT_NAME} PUBLIC
${SIMPLE_API_MAIN_DIR}/SimpleAPI)
add_dependencies(${PROJECT_NAME} ${SimpleAPI_lib_name})
```

Через использование технологии find_package

Для использования этого варианта должна быть установлена библиотека `pkg-config`:

```
sudo apt-get install pkg-config # пример для Ubuntu
```

Пример кода CMake

```
set(SIMPLE_API_MAIN_DIR ${CMAKE_CURRENT_LIST_DIR}/../..) # указать путь до
базовой директории библиотеки
include(${SIMPLE_API_MAIN_DIR}/utils.cmake) # для доступа к макросам библиотеки
list(APPEND CMAKE_MODULE_PATH "${SIMPLE_API_MAIN_DIR}/cmake/find_package/") #
для поиска модулей Find*.cmake

# код для создания таргета основного приложения
add_executable(...)

# предполагается, что имя таргета совпадает с именем проекта (PROJECT_NAME)
if(CMAKE_BUILD_TYPE STREQUAL "Release")
    find_package(SimpleAPI REQUIRED)
    if(SimpleAPI_FOUND)
        target_link_libraries(${PROJECT_NAME} PUBLIC SimpleAPI)
        target_include_directories(${PROJECT_NAME} PUBLIC
${SimpleAPI_INCLUDE_DIRS})
    else()
        message(FATAL "SimpleAPI not found")
    endif()
endif()
```

```
endif(SimpleAPI_FOUND)
else()
  find_package(SimpleAPI REQUIRED)
  if(SimpleAPI_FOUND)
    target_link_libraries(${PROJECT_NAME} PUBLIC SimpleAPI)
    target_include_directories(${PROJECT_NAME} PUBLIC
${SimpleAPI_INCLUDE_DIRS})
  else()
    message(FATAL "SimpleAPI not found")
  endif(SimpleAPI_FOUND)
endif()
```

3.2. Интеграция с QMake

Ввиду ограниченности системы сборки QMake, реализовано только самое базовое включение SimpleAPI. Проект SimpleAPI не будет собираться как самостоятельная библиотека, будь то статичная или динамическая.

Для добавления в ваш QMake-проект SimpleAPI следует придерживаться такого шаблона:

(теоретический project.pro)

```
QT = core

CONFIG += c++11 cmdline

include(SimpleAPI_library/SimpleAPI.pri) # добавление SimpleAPI как источник
исходников

# продолжение описания исходников основного приложения
SOURCES += \
    main.cpp
```


3.3 Упаковка скомпилированной библиотеки в .deb пакет

После сборки CMake-проекта библиотеки пользователь может выполнить команду для архивации файлов библиотеки в самостоятельный пакет установки:

```
# переходим в папку билда библиотеки SimpleAPI
cd build
# вызов команды-упаковщика
make package
```

Настроенные пути установки пакета предполагают расположение библиотеки по пути `/opt/SimpleAPI`. Упакованный .deb пакет не имеет сторонних зависимостей.

4. Класс Config

Используется как основной класс, с которым взаимодействует пользователь библиотеки.

Конструкторы Config позволяют создавать объекты Config на основе разрешённых выбранным стандартом значений (long double, bool, std::string, const char*, Config(ValueType::eJson), Config(ValueType::eArray), **TODO: Config(ValueType::XML)**, **TODO: Config(ValueType::YAML)**).

Форматы INI и JSON свободно конвертируются один в другой.

Используйте файл Config.h как источник актуальной информации о доступных методах. Обращайте внимание на отметки API_ в конце объявлений методов

Класс Config предоставляет следующие возможности:

1. методы работы с комментариями (информацию об их специфике смотрите в соответствующем разделе)
2. методы работы с комментариями элементов внутри контейнеров (имя метода пишется через знак подчёркивания '_')
3. методы установки значения - используйте методы setValue() или operator=()
4. методы получения реализации значений: getBool(), getNumber() (есть методы для прямого преобразования чисел к нужному формату, например, getUInt32_t()), getString())
5. методы работы с Config как контейнером (имя метода пишется через знак подчёркивания '_'), например, get_at(), get_front(), get_bool_at() и т.д.
6. методы работы с контейнерами Config как набором итераторов: getRange() и getMapRange() для соответствующих типов контейнеров
7. операторы сравнения одиночных объектов Config (опционально проверяются комментарии), а также сравнение контейнеров Config (опционально задаётся проверка сортировки элементов)

8. парсинг и запись JSON документов и файлов: `toString()` с указанием формата вывода
9. парсинг и запись INI документов и файлов: `toString()` с указанием формата вывода
10. если парсинг завершился с ошибкой, то метод `error()` вернёт сам факт ошибки, а метод `getError()` даст строку с описанием ошибки
11. методы очистки значений `clear()` и `clearContainer()` (не удаляет сам контейнер, но очищает его содержимое)
12. методы добавления значений в контейнер: `insert_*`() и `push_*`()
13. методы удаления значений из контейнеров: `erase_*`() и `pop_*`()
14. методы получения константы значения с последующим удалением из контейнера: `get_and_pop_*`()
15. методы получения информации о параметрах объекта: `getType()`, `is*()`, `isEqual()` и т.п.
16. методы получения количества элементов внутри `Config()`: контейнеры дадут актуальное количество элементов внутри, `eNull` даст 0, другие типы - 1
17. методы определения наличия ключа или значения внутри контейнера: `containsValue()` и `containsKey()`
18. операторы присвоения (аналог `setValue()`)
19. методы работы с файлами для чтения и записи конфигов

4.1. Формат JSON

Реализация, по большей части, соответствует стандарту RFC 8259.

В рамках SimpleAPI имеет тип `ValueType::eJson`.

ВАЖНО: автоматическая сортировка значений в SimpleAPI не предусмотрена.

Чтение конфига

Чтение организовано по принципу чтения по стандарту, с учётом поправок на условности формата:

- возможно чтение нескольких типов значений: число (long double), логический флаг (true, false, +, -), строка, пустое значение (null), массив и вложенный Json. Любое другое значение будет воспринято как строка (если не нарушает синтаксис указания значения)
- ключ с пустым значением воспринимается как null-элемент
- кавычки нужны только если строка ключа или значения содержит несколько слов
- перенос строки тоже считается разделителем, запятая необязательна
- в случае ошибки при чтении любой части документа успешно прочитанные значения не обнуляются, но заполняется описание ошибки
- метод `Config::error()` вернёт флаг наличия ошибки; текстовое описание ошибки доступно через вызов метода `Config::getError()`

Запись конфига

Запись Json происходит строго по стандарту (если не учитывать пользовательские комментарии - по умолчанию выключены) для совместимости с другими парсерами при передаче по сети.

Примеры кода

Примеры работы с Config как JSON-объектом рекомендуется изучать через

написанные юнит-тесты `test_json.cpp` и `test_json_array.cpp`.

4.2. Формат INI

Формат INI не имеет принятого стандарта и реализуется всеми по-разному. SimpleAPI в этом плане старается брать лучшее из других реализаций.

Чтение и запись комментариев более подробно будут рассмотрены в следующих главах.

В рамках SimpleAPI так же имеет тип `ValueType::eJson`. Полностью совместим с форматом Json, но не рекомендуется записывать в файл массив Json (`ValueType::eArray`) в формате INI, т.к. обратное чтение даст в результате иную структуру документа.

Чтение конфига

Используется принцип "одна строка - одно значение".

Ключ может быть большой вложенности. Разделителем вложенностей ключа является слэш '/' и обратный слэш '\'.

Если переменная занимает несколько строк, то участвуют следующие правила проверки. Переменная не прочитана полностью, если выполняется хотя бы одно из правил:

- при чтении значения переменной не закрыта кавычка
- текущая строка заканчивается на знак обратного слэша '\'
- следующая строка начинается с пробела или табуляции

Каждое значение будет проверено на тип внутренним парсером.

По идее, в формате INI хранятся только строки, но SimpleAPI расширяет эту идею до способности самостоятельно решать, что это за тип данных, будь то число (long double), JSON, массив в формате JSON, bool, пустое значение (null) или обычная строка (всё то же, что и в формате JSON). При этом для строк наличие кавычек необязательно.

Пробелы в начале и конце значения при обработке будут проигнорированы.

Формат INI подразумевает, что на верхнем уровне документа может существовать несколько групп. Для примера, QSettings от фреймворка Qt начальной группе

назначает имя "MAIN". SimpleAPI для этих целей использует пустой ключ "". Наличие названия группы полностью закрывает предыдущую группу. Наличие пустых строк или их отсутствие между группами не несёт логики.

Если в конфиге в рамках одной группы указано несколько одинаковых ключей, то их значения в совокупности являют собой массив с ключом в качестве имени массива.

Запись конфига

Логика записи итогового JSON-объекта в формате INI следующая:

- будут записаны по очереди BCE одиночные значения
- массивы будут записаны по очереди с одним и тем же ключом
- JSON-объекты первого уровня образуют группы значений, в рамках которых все остальные внутренние контейнеры записываются как многосоставной ключ с разделителем имени через слэш '/'

Примеры кода

Примеры работы с Config как INI-документом рекомендуется изучать через написанные юнит-тесты `test_ini.cpp` и `test_ini_array.cpp`.

4.3. Формат XML (TODO)

4.4. Формат YAML (TODO)

5. Работа с комментариями

За работу с комментариями отвечают классы `Comment` и `CommentDesign`. В `CommentDesign` указываются правила для определения комментариев.

По умолчанию используется стиль комментариев C/C++, а то есть `// comment` для однострочных комментариев и `/* comment */` для многострочных комментариев.

Стиль написания комментариев определяет параметр `CommentDesign` корневого элемента.

За исключением форматов XML и YAML, библиотека позволяет указать стили для чтения и записи комментариев при работе с файлами. Форматы XML и YAML имеют заданный их стандартами стиль комментариев.

При парсинге заполняется `CommentDesign::opt_multiline_column_size`. Всегда.

В SimpleAPI работа с комментариями предполагает два вида комментариев:

1. комментарий ДО значения
2. комментарий ПОСЛЕ значения

Комментарий ДО значения

Несколько комментариев до значения будут восприняты парсерами как один многострочный комментарий.

Следует использовать методы `Config::setPrefixComment()` или более общий `Config::setComment()`.

Комментарий ПОСЛЕ значения

Триггером начала отслеживания завершающего комментария является конец значения.

Триггером конца завершающего комментария считается перенос строки.

Ширина колонки многострочного комментария после значения не влияет на вывод (попытка записать в одну строку, если нет '\n').

Логика сравнения объектов `CommentDesign`

Следует держать в уме тот факт, что нельзя однозначно сравнить два `CommentDesign` с разными `opt_multiline_column_size`.

Проблема исходит из того, что при попытке записать такой стиль на комментарий, который как самую максимальную строку принимает, например $N-1$ (N - ширина колонки), то следующий парсер, который будет читать из этого результата в своём результате увидит ширину колонки как $N-1$.

Используемые библиотекой обработчики

За запись комментариев отвечает внутренняя функция `tools::ToComment()`.

За парсинг отвечает функция `tools::FromComment()`:

- однострочный комментарий:
 - может обрамляться 1 или 2 символами
 - если второй символ в строке пробел, то в *CommentDesign* запишется первый элемент и 0
- многострочный комментарий:
 - может обрамляться 1 или 2 символами (3 символ указывается как замена 1 при закрытии блока комментариев)
 - правильную границу комментария определяет функция чтения конфига
 - здесь проверка на существование границы в конце будет опускаться

Однострочные комментарии (пример)

Пример добавления нового шаблона поиска:

```
CommentDesign cd;  
cd.online_comment_variants.push_back({'#', 0});
```

Вектор принимает `std::array<char, 2>`. Оба символа являют собой строку, которая обозначает начало однострочного комментария и действует до переноса строки. Если второй символ указан как число 0 (без кавычек), то начало комментария определяется строго одним символом.

Многострочные комментарии (пример)

Пример добавления нового шаблона поиска:

```
CommentDesign cd;  
cd.multiline_comment_variants.push_back({'|', '|', '|'});
```

Вектор принимает `std::array<char, 3>`. Суть описания похожа на однострочные комментарии за тем исключением, что наличие третьего символа обозначает замыкающую границу комментария.

Примеры использования:

- `{'|', '|', '|'} -> {| multiline comment |}`
- `{'|', 0, '|'} -> { multiline comment }`
- `{'|', '|', 0} -> {| multiline comment |{`

Важные замечания

with_comments:

Комментарии не будут прочитаны или записаны, если поле

`CommentDesign::with_comments` в составе объекта `CommentDesign` не равно `true`.

CommentDesign

Переменная объекта `CommentDesign` в составе `Config` будет обновлена и дополнена при чтении нового файла/входной строки конфига.

Колонка

Для многострочных комментариев можно ограничить ширину колонки. Строки будут обрезаны автоматически.

Для включения функции следует использовать поле `CommentDesign::opt_multiline_column_size`.

Значение 0 отключает параметр.

Рамка

Для многострочных комментариев можно указать стиль рамки, например:

```
CommentDesign cd;  
cd.opt_multiline_border = '*';
```

Выведет так:

```
/*  
 * comment *  
*/
```

Значение 0 отключает параметр.

Стиль многострочного комментария

Для многострочных комментариев без рамки вывод по умолчанию указывает начало и конец комментария на отдельных строках, например:

```
/*  
multiline  
comment  
*/
```

Поле `CommentDesign::opt_multiline_border_at_content_line=true` изменяет этот момент:

```
/* multiline  
   comment */
```

Итоговый вид комментариев в строке вывода

Комментарий можно указать ДО значения и ПОСЛЕ него.

Несколько комментариев ДО значения сформируют один большой комментарий.

Комментарий ПОСЛЕ значения применяется только в том случае, если после написания значения комментарий был начат до переноса строки.

Комментарий ПОСЛЕ значения может быть только одним:

```
{
    // много
    // комментариев
    // ДО значения
    key="value", // комментарий ПОСЛЕ значения
    /* ещё
    один пример */
    /* вторая часть второго примера */
    key2=12465 /* большой
                комментарий
                после значения
                */
}
```

Уникальная обработка колонки комментария ПОСЛЕ значения

При записи комментария ПОСЛЕ значения параметр

`CommentDesign::opt_multiline_column_size` не влияет на ширину колонки, если в комментарии нет хотя бы одного явного переноса строки (`\n`). В таком случае будет сделана попытка записать комментарий как однострочный (для лучшей читабельности итогового конфига).

5. Сетевое взаимодействие

Библиотека SimpleAPI предоставляет возможности по относительно простому добавлению взаимодействия между удалёнными программами посредством сокетов.

Вариантов несколько:

Ручное управление

Использовать удобную обёртку над сокетом - `UdpSocket` и работать через методы `Socket::sendRawMsg()` и `Socket::recvRawMsg()`.

SimpleAPI Network

Предоставить библиотеке право самой управлять низкоуровневой логикой.

Пользователю нужно будет только вызывать метод отправки и callback, который библиотека будет вызывать при получении нового сообщения.

6. Дополнительные функции-утилиты

В ходе разработки библиотеки было сформировано множество полезных функций, которые могут показаться полезными пользователю. Данная статья рассказывает о некоторых из них.

Пространство имён: `simpleapi::utils`.

1. CharInString

Сигнатура:

```
bool CharInString(const char ch, std::string symbols) noexcept
```

Суть:

Определяет, есть ли указанный символ `ch` среди строки `symbols`.

2. ToHexString

Сигнатура:

```
std::string ToHexString(const std::vector<uint8_t>& data) noexcept
```

Суть:

Преобразует вектор байтов в строку формата HEX.

3. FromHexString

Сигнатура:

```
std::vector<uint8_t> FromHexString(std::string str) noexcept
```

Суть:

Преобразует строку формата HEX в вектор байтов.

4. GetStringCharCount

Сигнатура:

```
size_t GetStringCharCount(const std::string &str, bool only_visible)
```

Суть:

Указывает количество символов в строке формата UTF-8. Опционально можно узнать количество видимых символов.

5. GetStringCharCount

Сигнатура:

```
std::string GetNormalizeString(const std::string &input) noexcept
```

Суть:

Соберёт новую строку без пробелов и переносов строк.

6. RemoveIllegalSpaces*

Сигнатуры:

```
void RemoveFrontIllegalSpaces(std::string& string) noexcept void  
RemoveEndIllegalSpaces(std::string& string) noexcept void  
RemoveIllegalSpaces(std::string& string) noexcept
```

Суть:

Удаляют пробелы и табуляции в начале/конце строки.

7. SplitWithoutColumned

Сигнатура:

```
SplittedLines SplitWithoutColumned(const std::string& input_string) noexcept
```

Суть:

Обрежет входную строку на список строк (SplittedLines). Учитываются только пользовательские переносы строк.

8. SplitToColumns

Сигнатура:

```
SplittedLines SplitToColumns(const std::string& input_string, const size_t column_size)
noexcept
```

Суть:

Обрежет строку на подстроки с заданной шириной.

Если хотя бы одна строка неделима и превышает предел, то остальные строки будут выровнены по новому пределу.

Многоточие считается частью слова и не переносится на другую строку.

Пользовательские переносы строк будут сохранены.

9. VStringToString

Сигнатура:

```
std::string VStringToString(const VString& input_vec, const bool need_quotes = false)
noexcept
```

Суть:

Преобразует вектор строк в одну строку, при этом изначальные строки будут соединены знаком переноса '\n'.

10. SplitString

Сигнатура:

```
SplittedLines SplitString(const std::string input_string, char split_char, bool
with_empty_strings = true) noexcept
```

Суть:

Разделяет строку на подстроки с указанным символом-разделителем.